

```

timescale 1ns / 1ps
`default_nettype none
`include "imm_gen.v"
`include "parameters.vh"

module tb_imm_gen;

    // Inputs
    reg clk;
    reg rst_n;
    reg [31:0] i_Instr;

    // Outputs
    wire [31:0] imm_data;

    // Instantiate DUT (Device Under Test)
    imm_gen dut (
        .clk(clk),
        .rst_n(rst_n),
        .i_Instr(i_Instr),
        .imm_data(imm_data)
    );

    // Clock Generation
    always #5 clk = ~clk; // 10ns period (100MHz)

    // Test Procedure
    initial begin
        // Initialize signals
        clk = 0;
        rst_n = 0;
        i_Instr = 0;

        // Reset the DUT
        #10 rst_n = 1;

        // I-Type Instruction (ADDI)
        i_Instr = 32'b000000000101_00001_000_00010_0010011; // ADDI x2, x1, 5
        #10 $display("I-Type | Instruction: %b | Immediate: %d", i_Instr, imm_data);

        // Load (LD) Instruction
        i_Instr = 32'b0000000001010_00001_010_00010_0000011; // LD x2, 10(x1)
        #10 $display("LD | Instruction: %b | Immediate: %d", i_Instr, imm_data);

        // Store (S-Type) Instruction (SW)
        i_Instr = 32'b0000000001010_00010_010_00001_0100011; // SW x2, 10(x1)
    end

```

```

#10 $display("S-Type | Instruction: %b | Immediate: %d", i_Instr, imm_data);

// Branch (B-Type) Instruction (BEQ)
i_Instr = 32'b00000000_00010_00001_000_00000_1100011; // BEQ x1, x2, 8
#10 $display("B-Type | Instruction: %b | Immediate: %d", i_Instr, imm_data);

// Jump (J-Type) Instruction (JAL)
i_Instr = 32'b0000000000100000000000001010001111; // JAL x2, 4
#10 $display("J-Type | Instruction: %b | Immediate: %d", i_Instr, imm_data);

// Upper Immediate (U-Type) Instruction (LUI)
i_Instr = 32'b0000000000000000000000001010110111; // LUI x2, 0x00000
#10 $display("U-Type | Instruction: %b | Immediate: %d", i_Instr, imm_data);

// **Atomic Operation (AMOADD.W)**
i_Instr = 32'b00000_00010_00001_010_00010_0101111; // AMOADD.W x2, x1, x2
#10 $display("AMO | Instruction: %b | Immediate: %d", i_Instr, imm_data);

// **Atomic Operation (AMOSWAP.W)**
i_Instr = 32'b00001_00010_00001_010_00010_0101111; // AMOSWAP.W x2, x1, x2
#10 $display("AMO | Instruction: %b | Immediate: %d", i_Instr, imm_data);

// Stop simulation
#10 $finish;

end
endmodule

```